🧱 TiDPI 03 — Tablice o stałym i dynamicznym rozmiarze: stos, sterta i uczciwe porównanie

📋 Tematy do dziennika

```
[TiDPI] Tablice o rozmiarze stałym i dynamicznym – czas życia obiektu, stos, sterta oraz bezpieczne zarządzanie pamięcią w C++
[TiDPI] Testy funkcjonalne pamięci i danych – rozmiar, wartości graniczne, zgodność wyniku oraz typowe błędy wskaźników
[TiDPI] Porównawczy test wydajności przetwarzania i alokacji pamięci – poprawna metoda, pomiary, ograniczenia i raportowanie wyników
```

🎯 Cel bloku

Po tych zajęciach będziecie potrafili odróżnić tablicę o rozmiarze znanym podczas kompilacji od pamięci przydzielanej podczas działania programu, wyjaśnić rolę stosu i sterty, bezpiecznie zarządzać tablicą dynamiczną za pomocą RAII oraz zaprojektować test funkcjonalny i wydajnościowy, który nie miesza kosztu alokacji z kosztem przetwarzania danych.

Przeprowadzimy eksperyment odpowiadający na pytanie:

> Czy czas wykonania tej samej operacji różni się między lokalną tablicą, globalną tablicą i tablicą przydzieloną dynamicznie?

To pytanie jest celowo podchwytliwe. Po przygotowaniu pamięci wszystkie trzy zbiory mogą składać się z elementów tego samego typu ułożonych jeden za drugim. Różnić może się sposób przydzielenia pamięci, czas życia, dostępny rozmiar i sposób zarządzania. Nie wolno jednak automatycznie przypisać każdej różnicy w czasie słowom „stos”, „dane globalne” albo „sterta”.

> Pamięć nie czyta etykiety na pudełku. Procesor dostaje adres i wykonuje instrukcje. Naszym zadaniem jest ustalić, co naprawdę zmieniliśmy w eksperymencie.

🧩 Najpierw porządkujemy język

W rozmowach programistów często usłyszycie „tablica statyczna” i „tablica dynamiczna”. Te określenia bywają użyteczne, ale mogą ukrywać trzy różne pytania:

- Czy rozmiar tablicy jest znany podczas kompilacji, czy dopiero podczas działania programu?
- Jak długo istnieje obiekt?
- Gdzie i w jaki sposób została przydzielona jego pamięć?

Spójrzcie na cztery deklaracje:

```
int global_values[1000];

void example(std::size_t size)
{
    int local_values[1000];
    static int persistent_values[1000];
    int* dynamic_values = new int[size];

    delete[] dynamic_values;
}
```

`global_values` zadeklarowano poza funkcjami, dlatego jest **zmienną globalną**: jej nazwa należy do zakresu przestrzeni nazw. Tablica istnieje przez cały czas działania programu, ma więc statyczny czas przechowywania. Nie znajduje się na stosie; w typowej implementacji trafia do obszaru danych statycznych programu.

`local_values` ma rozmiar znany podczas kompilacji i automatyczny czas życia. W typowej implementacji lokalna tablica znajduje się w ramce stosu funkcji.

`persistent_values` również ma rozmiar znany podczas kompilacji, ale słowo kluczowe `static` nadaje jej statyczny czas przechowywania. Nie jest zwykłą lokalną tablicą na stosie: istnieje przez cały czas działania programu.

`dynamic_values` otrzymuje rozmiar podczas działania programu. Operator `new[]` przydziela pamięć dynamiczną, zwykle z obszaru nazywanego stertą, a `delete[]` ją zwalnia.

Dlatego w tej lekcji będziemy używali precyzyjniejszych określeń:

- **lokalna tablica o stałym rozmiarze**;
- **globalna tablica o stałym rozmiarze i statycznym czasie przechowywania**;
- **obiekt o statycznym czasie przechowywania**;
- **tablica przydzielana dynamicznie**.

Deklaracja	Zakres widoczności	Czas życia	Typowe miejsce przechowywania
lokalna tablica `int values[1000];`	bieżący blok lub funkcja	do opuszczenia zakresu	stos
lokalna tablica `static int values[1000];`	bieżący blok lub funkcja	cały czas działania programu	dane statyczne
globalna tablica `int values[1000];`	zakres przestrzeni nazw	cały czas działania programu	dane statyczne
tablica utworzona przez `new int[size]`	dostęp przez przechowywany wskaźnik	do zwolnienia pamięci	sterta

Pojęcia **globalna** i **statyczny czas przechowywania** nie są synonimami. Pierwsze mówi przede wszystkim o miejscu deklaracji i zakresie nazwy, drugie o czasie istnienia obiektu. Zmienna globalna ma statyczny czas przechowywania, ale lokalna zmienna ze słowem `static` również ma statyczny czas przechowywania, choć jej nazwa pozostaje lokalna.

Warto zauważyć jeszcze jedną pułapkę:

```
int* global_pointer = new int[1000];
```

Sam `global_pointer` jest zmienną globalną o statycznym czasie przechowywania, ale wskazywana tablica została przydzielona dynamicznie na stercie. Wskaźnik i obiekt, na który wskazuje, mogą mieć inne miejsce oraz inny sposób zarządzania czasem życia.

> Zmienna globalna jest jak tablica ogłoszeń na szkolnym korytarzu: istnieje przez cały dzień i wiele osób może do niej dotrzeć. To bywa wygodne podczas demonstracji, ale nadmiar wspólnego, modyfikowalnego stanu szybko prowadzi do pytania: „Kto znowu zmienił ten wynik?”.

W zwykłym projekcie ograniczamy niepotrzebne zmienne globalne, ponieważ utrudniają śledzenie, kto i kiedy zmienił dane. W tej lekcji użyjemy jednej globalnej tablicy świadomie, aby porównać trzy sposoby przechowywania danych.

> Każdy jamnik jest psem, ale nie każdy pies jest jamnikiem. Podobnie: lokalna tablica o stałym rozmiarze często trafia na stos, ale słowo `static` w C++ oznacza coś bardziej konkretnego niż „rozmiar wpisany na sztywno”.

⚠️ C++ nie ma standardowych lokalnych tablic o rozmiarze podanym w czasie działania

Rozmiar zwykłej lokalnej tablicy powinien być stałym wyrażeniem:

```
constexpr std::size_t size = 1000;
int values[size];
```

Taki zapis nie jest natomiast przenośnym standardowym C++:

```
std::size_t size = 0;
std::cin >> size;
int values[size]; // nie jest standardową lokalną tablicą C++
```

Niektóre kompilatory mogą dopuszczać podobny zapis jako rozszerzenie. W Visual Studio nie należy na nim opierać programu. Jeśli rozmiar poznajemy podczas działania, używamy na przykład `std::vector` albo pamięci dynamicznej zarządzanej przez `std::unique_ptr`.

Stos — porządek, szybkość i ograniczony rozmiar

Stos (*stack*) to obszar pamięci związany między innymi z wywołaniami funkcji. Gdy funkcja zostaje wywołana, powstaje jej ramka przechowująca potrzebne informacje i część zmiennych lokalnych. Po zakończeniu funkcji ramka znika automatycznie.

Można wyobrazić go sobie jako stos tac w stołówce: ostatnia odłożona taca zostanie zabrana jako pierwsza. Taki porządek nazywamy **LIFO** — *Last In, First Out*.

Typowe zalety:

- automatyczne zakończenie czasu życia lokalnych obiektów;
- bardzo mały narzut utworzenia i usunięcia ramki funkcji;
- prosty, uporządkowany sposób zarządzania.

Najważniejsze ograniczenie: stos ma ograniczony rozmiar. Bardzo duża lokalna tablica albo zbyt głęboka rekurencja może doprowadzić do **przepełnienia stosu** (*stack overflow*) i zakończenia programu.

> Stos jest szybki i uporządkowany, ale nie jest magazynem bez dna. Próba wniesienia do podręcznej szafki całej serwerowni zwykle kończy się problemem logistycznym.

Nie będziemy przyjmowali jednej „magicznej” granicy liczby elementów. Rozmiar stosu może zależeć od systemu, ustawień linkera, architektury, konfiguracji programu, innych zmiennych lokalnych i sposobu wygenerowania kodu.

Sterta — elastyczność i odpowiedzialność

Sterta (*heap*) to obszar wykorzystywany do dynamicznego przydzielania pamięci. Możemy poprosić o pamięć, gdy znamy już rozmiar danych, i zdecydować, jak długo ma istnieć.

W tradycyjnym zapisie C++ wygląda to tak:

```
std::size_t size = 1000;
int* values = new int[size];

// korzystanie z values

delete[] values;
```

Operator `delete[]` musi odpowiadać operatorowi `new[]`. Samo `delete values;` byłoby błędem.

Ręczne zarządzanie pamięcią łatwo prowadzi do problemów:

- **wyciek pamięci** (*memory leak*) — program traci możliwość zwolnienia przydzielonej pamięci;
- **wiszący wskaźnik** (*dangling pointer*) — wskaźnik nadal przechowuje adres obiektu, który już nie istnieje;

- **podwójne zwolnienie** (*double delete*) — program próbuje drugi raz zwolnić tę samą pamięć;
- dostęp poza zakresem tablicy — program odczytuje lub zapisuje pamięć, która do niej nie należy.

Te błędy mogą zakończyć program, uszkodzić dane albo przez długi czas pozostawać niewidoczne. Brak awarii nie jest dowodem poprawności.

> Wiszący wskaźnik przypomina zapisany adres hotelu po jego wyburzeniu. Kartka nadal istnieje, adres wygląda poważnie, ale próba wejścia do pokoju może zakończyć się dość nieprzenośnym wyjątkiem budowlanym.

♻️ RAII — zasób pilnowany przez obiekt

W nowoczesnym C++ unikamy „nagiego” `new` i `delete`, jeśli gotowy obiekt może bezpiecznie zarządzać pamięcią za nas. Służy temu zasada **RAII** — *Resource Acquisition Is Initialization*, czyli powiązanie czasu życia zasobu z czasem życia obiektu.

Dla dynamicznej tablicy możemy użyć inteligentnego wskaźnika `std::unique_ptr`:

```
#include <memory>

std::size_t size = 1000;
auto values = std::make_unique<int[]>(size);

values[0] = 42;
```

Gdy `values` wychodzi ze swojego zakresu, pamięć zostaje zwolniona automatycznie. Nie dopisujemy `delete[]`.

Jeszcze częściej używa się `std::vector`, który zna swój rozmiar, potrafi go zmieniać i również automatycznie zarządza pamięcią:

```
#include <vector>

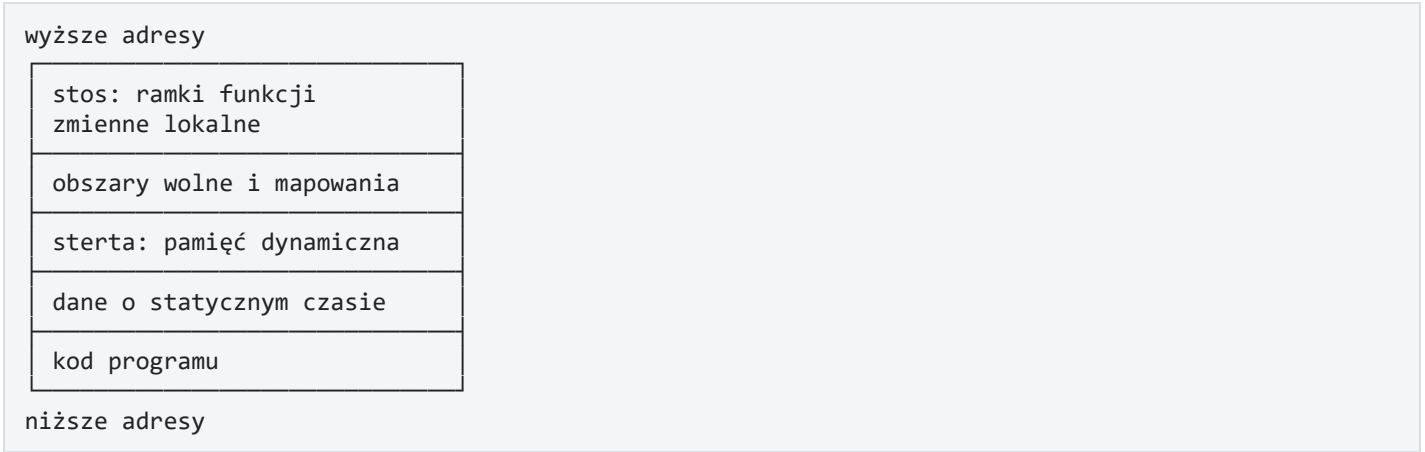
std::size_t size = 1000;
std::vector<int> values(size);
values[0] = 42;
```

W eksperymencie użyjemy `std::unique_ptr`, ponieważ chcemy wyraźnie obserwować dynamicznie przydzieloną tablicę. W zwykłym programie `std::vector` jest zazwyczaj wygodniejszym i bezpieczniejszym wyborem.

> RAII jest jak szatnia wydająca numerek: obiekt pamięta, który zasób odebrał, i oddaje go przy wyjściu. Ręczne `new` bez planu zwolnienia przypomina wyjście z kina w cudzym płaszczu — przez chwilę może się wydawać, że wszystko działa.

🗺️ Uproszczona mapa pamięci procesu

Poniższy schemat buduje intuicję, a nie przedstawia gwarantowanego planu adresów każdego programu:



Nie zakładajcie, że stos zawsze „rośnie w dół”, sterta zawsze „w górę”, a ich adresy muszą układać się dokładnie jak na rysunku. Tak często przedstawia się typową organizację procesu, ale standard C++ nie narzuca systemowi operacyjnemu konkretnego układu adresów.

🔍 Co właściwie testujemy?

W tej lekcji wykonamy dwa rodzaje testów dynamicznych, czyli testów przeprowadzanych przez uruchomienie badanego programu lub jego fragmentu. **Test funkcjonalny** odpowie na pytanie, czy program działa poprawnie i zwraca oczekiwane wyniki. **Test wydajnościowy** pozwoli porównać czas wykonania tej samej operacji w trzech wariantach przechowywania danych.

✅ Test funkcjonalny

Sprawdzamy, czy program:

- przygotował wszystkie trzy zbiory o oczekiwanym rozmiarze;
- zapisał i odczytał pierwszy oraz ostatni element;
- umieścił te same wartości we wszystkich zbiorach;
- po przetworzeniu zwrócił oczekiwane wartości.

Nie wystarczy sprawdzić, że każda liczba znajduje się w szerokim dozwolonym zakresie. Program mógłby zgubić lub zmienić wartość, a tak słaby test nadal powiedziałby „OK”.

⏱️ Test wydajnościowy

Porównujemy czas wykonania tej samej operacji na już istniejących danych:

- lokalnej tablicy o stałym rozmiarze;
- globalnej tablicy o stałym rozmiarze i statycznym czasie przechowywania;
- tablicy przydzielonej dynamicznie.

Alokację pamięci wykonujemy przed uruchomieniem stopera. Dzięki temu test przetwarzania nie odpowiada jednocześnie na dwa pytania. Koszt samej alokacji będzie osobnym, trudniejszym eksperymentem.

> Jeśli jeden zawodnik przed biegiem musi jeszcze zbudować sobie buty, nie mierzymy już wyłącznie szybkości biegania.

🧪 Projekt eksperymentu

We wszystkich trzech tablicach przechowamy ten sam typ danych: `std::uint32_t`. Tablice lokalna i globalna będą miały rozmiar ustalony podczas kompilacji, a dynamiczna otrzyma ten sam rozmiar podczas działania programu. Każda z nich będzie ciągłym obszarem pamięci.

W każdej próbie:

- przygotujemy trzy zbiory poza pomiarem;
- sprawdzimy ich początkową zgodność;
- wykonamy tę samą operację na wszystkich elementach;
- będziemy rotowali kolejność trzech pomiarów między próbami;
- sprawdzimy wynik względem prostej reguły oczekiwanej;
- zapiszemy surowe czasy wszystkich wariantów.

Funkcja `process_data` przejdzie po całej tablicy `rounds` razy. Podczas każdego przebiegu zwiększy każdy element o `1`, dlatego po zakończeniu jego oczekiwana wartość będzie równa wartości początkowej powiększonej o `rounds`. Wielokrotne przejście wydłuża badaną operację, dzięki czemu łatwiej zmierzyć jej czas.

💻 Kompletny program bazowy

Program jest przeznaczony dla Visual Studio 2022 i standardu C++17 lub nowszego. Domyślny rozmiar lokalnej tablicy to około 200 KiB danych, co powinno pozostawić zapas względem typowych ustawień stosu, ale nie jest gwarancją dla każdego środowiska.

```
#include <algorithm>
#include <array>
#include <chrono>
#include <cstdint>
#include <iomanip>
#include <iostream>
#include <memory>

constexpr std::size_t element_count = 50'000;
constexpr std::uint32_t rounds = 100;
constexpr int trial_count = 6;

using FixedArray = std::array<std::uint32_t, element_count>;

// Tablica globalna pozwala porównać ten sam typ danych przechowywany w innym
// obszarze pamięci i istniejący przez cały czas działania programu.
FixedArray global_values{};

template <typename Operation>
long long measure_microseconds(Operation operation)
{
// Interesuje nas czas trwania operacji, nie godzina. steady_clock jest
    // monotoniczny, więc zmiana zegara systemowego nie zafałszuje pomiaru.
    const auto start = std::chrono::steady_clock::now();
    operation();
    const auto stop = std::chrono::steady_clock::now();

    return std::chrono::duration_cast<std::chrono::microseconds>(stop - start).count();
}

void prepare_data(std::uint32_t* data, std::size_t size)
{
// Każdy wariant pamięci dostaje ten sam prosty wzorzec. Dzięki temu różnice
    // wynikają ze sposobu przechowywania, a poprawność łatwo sprawdzić.
    for (std::size_t i = 0; i < size; ++i)
    {
        data[i] = static_cast<std::uint32_t>(i % 1000);
    }
}

void process_data(std::uint32_t* data, std::size_t size)
{
// Pojedyncze dodanie trwa zbyt krótko, aby je wiarygodnie zmierzyć. Powtarzamy
    // operację wiele razy, żeby jej koszt był widoczny ponad szum pomiarowy.
    for (std::uint32_t round = 0; round < rounds; ++round)
    {
        for (std::size_t i = 0; i < size; ++i)
        {
            ++data[i];
        }
    }
}

bool verify_initial_data(const std::uint32_t* data, std::size_t size)
{
    if (data == nullptr || size == 0)
```

```
    {
        return false;
    }

    for (std::size_t i = 0; i < size; ++i)
    {
        const std::uint32_t expected = static_cast<std::uint32_t>(i % 1000);

        if (data[i] != expected)
        {
            return false;
        }
    }

    return true;
}

bool verify_processed_data(const std::uint32_t* data, std::size_t size)
{
    if (data == nullptr || size == 0)
    {
        return false;
    }

    for (std::size_t i = 0; i < size; ++i)
    {
        const std::uint32_t expected =
            static_cast<std::uint32_t>(i % 1000) + rounds;

        if (data[i] != expected)
        {
            return false;
        }
    }

    return true;
}

std::uint64_t checksum(const std::uint32_t* data, std::size_t size)
{
    std::uint64_t result = 0;

    for (std::size_t i = 0; i < size; ++i)
    {
        result += data[i];
    }

    return result;
}

struct TrialResult
{
    int trial;
    long long fixed_microseconds;
    long long global_microseconds;
    long long dynamic_microseconds;
    bool fixed_correct;
    bool global_correct;
    bool dynamic_correct;
    bool equal_results;
    std::uint64_t fixed_checksum;
    std::uint64_t global_checksum;
    std::uint64_t dynamic_checksum;
```

```
};

TrialResult run_trial(int trial)
{
// Tworzymy te obiekty razem, aby w jednej próbie porównać pamięć automatyczną
    // (lokalną) z pamięcią przydzielaną dynamicznie na stercie.
    FixedArray fixed_values{};
    auto dynamic_values = std::make_unique<std::uint32_t[]>(element_count);

// Badamy przetwarzanie elementów, a nie koszt tworzenia tablic. Dlatego
    // przydział pamięci i przygotowanie danych wykonujemy przed pomiarem.
    prepare_data(fixed_values.data(), fixed_values.size());
    prepare_data(global_values.data(), global_values.size());
    prepare_data(dynamic_values.get(), element_count);

    if (!verify_initial_data(fixed_values.data(), fixed_values.size()) ||
        !verify_initial_data(global_values.data(), global_values.size()) ||
        !verify_initial_data(dynamic_values.get(), element_count))
    {
// Nie mierzymy błędnych danych: szybkie wykonanie operacji na złym wejściu
        // nie jest dowodem ani poprawności, ani dobrej wydajności.
        return TrialResult
        {
            trial,
            0, 0, 0,
            false, false, false, false,
            0, 0, 0
        };
    }

    long long fixed_time = 0;
    long long global_time = 0;
    long long dynamic_time = 0;

// Nadajemy operacjom nazwy, aby zmieniać ich kolejność bez kopiowania kodu
    // pomiarowego i bez ryzyka, że warianty będą mierzone inaczej.
    const auto measure_fixed = [&fixed_values, &fixed_time]()
    {
        fixed_time = measure_microseconds([&fixed_values]()
        {
            process_data(fixed_values.data(), fixed_values.size());
        });
    };

    const auto measure_global = [&global_time]()
    {
        global_time = measure_microseconds([]()
        {
            process_data(global_values.data(), global_values.size());
        });
    };

    const auto measure_dynamic = [&dynamic_values, &dynamic_time]()
    {
        dynamic_time = measure_microseconds([&dynamic_values]()
        {
            process_data(dynamic_values.get(), element_count);
        });
    };

    if (trial % 3 == 1)
    {
// Rotacja kolejności rozdziela między warianty wpływ rozgrzania procesora,
```

```
        // pamięci podręcznej i chwilowych zadań działających w tle.
        measure_fixed();
        measure_global();
        measure_dynamic();
    }
    else if (trial % 3 == 2)
    {
        measure_global();
        measure_dynamic();
        measure_fixed();
    }
    else
    {
        measure_dynamic();
        measure_fixed();
        measure_global();
    }

    const bool fixed_correct =
        verify_processed_data(fixed_values.data(), fixed_values.size());
    const bool global_correct =
        verify_processed_data(global_values.data(), global_values.size());
    const bool dynamic_correct =
        verify_processed_data(dynamic_values.get(), element_count);
    const bool equal_results =
// Dwie różne tablice mogą przypadkiem mieć tę samą sumę kontrolną. Dlatego
        // poprawność potwierdzamy także przez porównanie każdego elementu.
        std::equal(
            fixed_values.begin(),
            fixed_values.end(),
            global_values.begin()) &&
        std::equal(
            fixed_values.begin(),
            fixed_values.end(),
            dynamic_values.get());

// Sumy kontrolne zapisujemy jako krótki ślad wyniku, ale nie zastępują one
    // dokładnej walidacji wykonanej element po elemencie.
    return TrialResult
    {
        trial,
        fixed_time,
        global_time,
        dynamic_time,
        fixed_correct,
        global_correct,
        dynamic_correct,
        equal_results,
        checksum(fixed_values.data(), fixed_values.size()),
        checksum(global_values.data(), global_values.size()),
        checksum(dynamic_values.get(), element_count)
    };
}

int main()
{
    std::cout
        << "element_count=" << element_count << '\n'
        << "bytes_per_array=" << element_count * sizeof(std::uint32_t) << '\n'
        <<
"trial;fixed_us;global_us;dynamic_us;fixed_ok;global_ok;dynamic_ok;equal;fixed_checksum;global_che
cksum;dynamic_checksum\n";
```

```
    bool all_correct = true;

    for (int trial = 1; trial <= trial_count; ++trial)
    {
        const TrialResult result = run_trial(trial);

        all_correct =
            all_correct &&
            result.fixed_correct &&
            result.global_correct &&
            result.dynamic_correct &&
            result.equal_results;

        std::cout
            << result.trial << ';'
            << result.fixed_microseconds << ';'
            << result.global_microseconds << ';'
            << result.dynamic_microseconds << ';'
            << std::boolalpha << result.fixed_correct << ';'
            << result.global_correct << ';'
            << result.dynamic_correct << ';'
            << result.equal_results << ';'
            << result.fixed_checksum << ';'
            << result.global_checksum << ';'
            << result.dynamic_checksum << '\n';
    }

    std::cout << "all_correct=" << std::boolalpha << all_correct << '\n';
    return all_correct ? 0 : 1;
}
```

🧭 Jak czytać program?

`std::array` przechowuje elementy bezpośrednio wewnątrz swojego obiektu. Obiekt `fixed_values` jest lokalny w funkcji `run_trial`, dlatego w typowej kompilacji jego dane znajdą się w ramce stosu.

`global_values` jest obiektem tego samego typu `FixedArray`, ale zadeklarowanym poza funkcjami. Ma statyczny czas przechowywania i w typowej implementacji znajduje się w obszarze danych statycznych programu. Przed każdą próbą program ponownie wypełnia tę tablicę, aby nie odziedziczyła przetworzonych wartości z poprzedniej próby.

`std::make_unique<std::uint32_t[]>` przydziela tablicę dynamicznie i zwraca inteligentny wskaźnik. Metoda `.get()` udostępnia surowy adres funkcjom, ale własność pamięci nadal pozostaje w `std::unique_ptr`.

Funkcje `prepare_data` i `verify_initial_data` działają przed pomiarem. Funkcja `process_data` jest jedyną operacją objętą stoperem. Po pomiarze `verify_processed_data` sprawdza każdy element względem oczekiwanej wartości, a `std::equal` porównuje wszystkie wyniki.

Sześć prób pozwala dwukrotnie wykonać każdą z trzech rotacji kolejności: lokalna–globalna–dynamiczna, globalna–dynamiczna–lokalna oraz dynamiczna–lokalna–globalna. Dzięki temu żaden wariant nie jest zawsze pierwszy ani zawsze ostatni.

Suma kontrolna, czyli `checksum`, stanowi dodatkowy, łatwy do zapisania dowód wykorzystania danych. Sama zgodność sum kontrolnych nie dowodzi równości tablic — różne dane mogą mieć tę samą sumę — dlatego program wykonuje również kontrolę każdego elementu.

Rozmiar jednej tablicy program wypisuje w bajtach. Aby przeliczyć go w przybliżeniu na kibibajty:

```
KiB = liczba bajtów / 1024
```

> Kibibajt (KiB) oznacza 1024 bajty. Kilobajt (kB) w układzie SI oznacza 1000 bajtów. W codziennych rozmowach skróty bywają mieszane, ale w raporcie warto napisać dokładnie, której jednostki używacie. Pamięć lubi precyzję, nawet jeśli marketing dysków czasem lubi większe liczby.

✅ Co wolno wywnioskować z wyników?

Jeżeli trzy warianty osiągają podobne czasy, jest to sensowny rezultat. Po przygotowaniu pamięci każda funkcja przechodzi po ciągłych elementach tego samego typu i wykonuje ten sam kod.

Jeżeli jedna wersja okaże się szybsza w pojedynczej próbie, nie ogłaszajcie jeszcze zwycięzcy. Sprawdźcie wszystkie powtórzenia i rozstęp wyników. Różnica może wynikać z zakłóceń, kolejności wykonania, położenia pamięci, optymalizacji albo innych czynników.

Nie wolno na podstawie tego doświadczenia napisać:

> „Sterta jest zawsze wolniejsza od stosu”.

Można natomiast napisać:

> „W naszym eksperymencie przetwarzanie lokalnej, globalnej i dynamicznej ciągłej tablicy miało podobny czas. Nie mierzyliśmy w tej serii kosztu dynamicznego przydzielania pamięci”.

Albo, jeśli dane rzeczywiście to pokażą:

> „W opisanych warunkach zaobserwowaliśmy powtarzalną różnicę, ale eksperyment nie pozwala przypisać jej wyłącznie miejscu przydzielenia pamięci”.

🖥️ x86 i x64 — nie organizujemy wyścigu na same nazwy

W konfiguracji x86 wskaźnik ma zwykle 32 bity, a w x64 — 64 bity. Proces 64-bitowy ma znacznie większą przestrzeń adresową, a architektura x64 udostępnia również inny zestaw możliwości procesora i reguł wywołań. Nie wynika z tego jednak, że każdy program x64 działa szybciej.

Większe wskaźniki mogą zwiększać rozmiar struktur zawierających wiele adresów. Z drugiej strony kod x64 może korzystać z większej liczby rejestrów i innych możliwości. O wyniku decydują konkretny kod, kompilator, ustawienia oraz dane.

Porównanie x86 i x64 będzie rozszerzeniem. Uczciwy raport nie napisze „64 jest dwa razy większe, więc program jest dwa razy szybszy”. Matematyka bywa bezlitosna dla marketingu, ale jeszcze bardziej dla takiego wniosku.

☠️ Błędy pamięci, które warto umieć rozpoznać

Poniższe fragmenty są celowo błędne. Nie kopiujcie ich do programu bazowego.

💧 Wyciek pamięci

```
void leak()
{
    int* values = new int[1000];
    values[0] = 42;
    // brak delete[] values;
}
```

Po zakończeniu funkcji znika wskaźnik, ale przydzielony obszar nie został zwolniony.

🪝 Wiszący wskaźnik i użycie po zwolnieniu

```
int* values = new int[1000];
delete[] values;
values[0] = 42; // błąd: pamięć już nie należy do tablicy
```

Przypisanie `nullptr` po `delete[]` może pomóc uniknąć przypadkowego ponownego użycia jednego wskaźnika, ale nie naprawia innych kopii tego samego adresu. Lepszą ochroną jest takie projektowanie programu, aby własność była jednoznaczna i pilnowana przez RAII.

🚧 Wyjście poza zakres

```
int values[10]{};
values[10] = 42; // poprawne indeksy to 0–9
```

Program może nie zakończyć się natychmiast. To nadal błąd. Zapis poza zakresem powoduje niezdefiniowane zachowanie, czyli standard C++ nie określa, co wydarzy się dalej.

🧪 Ćwiczenia i eksperyment

Wykonujcie poziomy kolejno. Wyższy poziom zakłada poprawne ukończenie niższych. Wszystkie pomiary wydajności wykonujcie w konfiguracji Release bez debuggera, a obserwacje pamięci — w konfiguracji Debug.

🟢 Poziom A — rozumiem rodzaje pamięci i sprawdzam funkcjonalność

- Uruchomcie kompletny program bazowy.
- Obliczcie ręcznie liczbę bajtów i kibibajtów zajmowanych przez jedną tablicę, a następnie porównajcie wynik z komunikatem programu.
- W debuggerze odszukajcie adresy `fixed_values.data()`, `global_values.data()` oraz `dynamic_values.get()`. Zapiszcie obserwację, ale nie zakładajcie, że konkretne adresy powtórzą się przy następnym uruchomieniu.
- Pokażcie dowody przejścia kontroli danych początkowych, danych wynikowych i zgodności trzech tablic.
- Zmieńcie celowo oczekiwaną wartość w `verify_processed_data`, zaobserwujcie wykrycie błędu, a następnie przywróćcie poprawny kod.
- Wyjaśnijcie różnicę między lokalną tablicą o stałym rozmiarze, globalną tablicą, lokalnym obiektem zadeklarowanym ze słowem `static` i tablicą przydzielaną dynamicznie.
- Wyjaśnijcie, w jaki sposób `std::unique_ptr` realizuje zasadę RAII.

🛡️ Poziom B — porównuję przetwarzanie już przydzielonych danych

- Wykonajcie po sześć prób dla co najmniej trzech wartości `element_count`, na przykład `5'000`, `20'000` i `50'000`. Zmiana tej stałej wymaga ponownej kompilacji programu.
- Dla każdego rozmiaru zachowajcie surowe czasy trzech wariantów, minimum, maksimum i rozstęp.
- Nie zmieniajcie `rounds` w środku jednej serii.
- Sprawdźcie, czy wszystkie testy funkcjonalne zakończyły się powodzeniem.
- Przygotujcie wykres czasu przetwarzania względem liczby elementów.
- Oceńcie, czy wyniki pozwalają wskazać stabilną przewagę któregoś wariantu. Oddzielcie obserwację od przypuszczenia o przyczynie.

Ukończenie wyłącznie poziomów A i B pozwala uzyskać najwyżej ocenę 4.

🔧 Poziom C — rozdzielam kolejne pytania badawcze

Wykonajcie dwa z trzech poniższych eksperymentów:

- **Koszt alokacji dynamicznej.** Zaprojektujcie osobny pomiar `std::make_unique<std::uint32_t[]>` wraz ze zwolnieniem pamięci. Nie dodawajcie go do czasu `process_data`. Wykonajcie wiele powtórzeń i opiszcie, co dokładnie obejmuje stoper.
- **Granica lokalnej tablicy.** Stopniowo zwiększajcie `element_count`, za każdym razem zapisując rozmiar w bajtach. Pracujcie na kopii projektu i nie ustawiajcie od razu absurdalnej wartości. Zanotujcie największy uruchomiony wariant oraz pierwszy wariant zakończony niepowodzeniem. Nie przedstawiajcie otrzymanej granicy jako uniwersalnej.
- **x86 i x64.** Wykonajcie tę samą serię w Release dla obu platform. Sprawdźcie `sizeof(void*)`, zachowajcie identyczny kod i warunki oraz oddzielcie różnicę rozmiaru wskaźnika od wniosku o czasie działania.

Dodajcie do raportu hipotezę każdego wybranego badania, procedurę, surowe wyniki, kontrolę poprawności oraz ograniczenia. Poprawne ukończenie poziomów A–C otwiera drogę do oceny 5.

> Eksperyment z granicą stosu celowo może zakończyć badaną kopię programu. Zapiszcie pracę przed uruchomieniem, nie trzymajcie w programie niezapisanych danych i nie próbujcie wywoływać podobnych awarii w cudzych procesach. Testujemy własny program, nie cierpliwość administratora pracowni.

🚀 Poziom D — własne badanie ponadprogramowe

Zaprojektujcie jedno rozszerzenie pokazujące wiedzę wykraczającą poza rdzeń lekcji. Przykładowe kierunki:

- porównanie surowej tablicy dynamicznej, `std::unique_ptr` i `std::vector` z poprawnym rozdzieleniem alokacji od przetwarzania;
- demonstracja, w której RAII poprawnie zwalnia zasób mimo wcześniejszego wyjścia z funkcji lub zgłoszenia wyjątku;
- użycie dostępnego w Visual Studio narzędzia diagnostycznego do wykrycia kontrolowanego wycieku we własnym małym programie;
- zbadanie wpływu rozmiaru elementu lub struktury na zużycie pamięci i czas przejścia po danych;
- inne pytanie uzgodnione z prowadzącym.

Samo użycie trudniejszej nazwy lub narzędzia nie zapewnia oceny 6. Musicie wyjaśnić mechanizm, przygotować kontrolę poprawności, pokazać powtarzalne dowody i obronić swoje wnioski.

📝 Sprawozdanie

Oddajecie jedno indywidualne sprawozdanie w Markdownie zgodne z zasadami lekcji 01. Sprawozdanie może zostać przekazane internetowemu agentowi AI do wstępnej analizy, dlatego musi być całkowicie zanonimizowane.

🔒 Obowiązkowa anonimizacja

- użyjcie wyłącznie przydzielonego kryptonimu TiDPI;
- nie umieszczajcie imienia, nazwiska, adresu e-mail ani nazwy konta w treści, nazwie pliku, metadanych, zrzutach ekranu, ścieżkach katalogów i innych dowodach;
- sprawdźcie kadrowanie zrzutów oraz właściwości plików eksportowanych do DOCX lub PDF;
- mapę `kryptonim TiDPI ↔ uczeń` przechowuje wyłącznie nauczyciel.

🧱 Wymagana struktura raportu

- `Kryptonim TiDPI i wykonany poziom;`
- `Cel oraz hipoteza;`
- `Środowisko, konfiguracja i rozmiary danych;`
- `Badane warianty i zmienne;`
- `Testy funkcjonalne;`
- `Procedura pomiarowa;`
- `Surowe wyniki;`
- `Opracowanie wyników i wykres;`
- `Interpretacja;`
- `Ograniczenia eksperymentu;`
- `Wnioski;`
- `Wykorzystanie AI i sposób weryfikacji.`

Każdy dowód oznaczcie identyfikatorem, na przykład `D01`, `D02`, `D03`, i przywołajcie go w odpowiedniej części raportu. Podpis ma informować, co przedstawia dowód i dlaczego jest potrzebny. Dzięki temu nauczyciel oraz wspierający go agent mogą wskazać konkretną podstawę analizy zamiast formułować ogólną opinię.

W sekcji dotyczącej AI zgodnie z prawdą podajcie narzędzie, zakres pomocy i sposób sprawdzenia propozycji. Samo użycie AI nie obniża oceny. Oddajecie jednak własną pracę i musicie rozumieć każdy fragment kodu, metodę, wynik oraz wniosek.

📊 Progi i kryteria oceny

Wyższy próg obejmuje wymagania wszystkich niższych progów. Tabela wskazuje maksymalną możliwą ocenę za poprawnie wykonany, udokumentowany i terminowo oddany zakres.

Maksymalna ocena	Wymagany zakres	Minimalne dowody
2	uruchomiony program i częściowe wykonanie poziomu A	komunikat programu, podstawowy opis trzech tablic i przynajmniej jedna kontrola funkcjonalna
3	poprawnie ukończony poziom A	obliczenie rozmiaru, trzy adresy z debuggera, test danych początkowych i wynikowych, rozróżnienie globalności od statycznego czasu przechowywania oraz wyjaśnienie RAII
4	poprawnie ukończone poziomy A i B	`3 rozmiary × 6 prób × 3 warianty`, surowe dane, minimum, maksimum, rozstęp, wykres i ostrożny wniosek
5	poziomy A–C	dwa oddzielne eksperymenty poziomu C, jawne hipotezy, kontrole poprawności, surowe wyniki i ograniczenia
6	poziomy A–C oraz wartościowe badanie poziomu D	wiedza ponadprogramowa, samodzielny projekt badania, powtarzalne dowody, krytyczna interpretacja i obrona pracy

Niezależnie od poziomu oceniane są:

- poprawność kodu i testów funkcjonalnych;
- rozdzielenie alokacji, przygotowania danych i mierzonej operacji;
- kompletność surowych wyników;
- uczciwe opisanie awarii i prób nieudanych;
- rozdzielenie obserwacji, interpretacji i przypuszczenia;
- czytelność, identyfikatory dowodów i anonimizacja;
- zrozumienie oraz umiejętność obrony pracy.

> Ocena nie rośnie od samego zużycia pamięci. Utworzenie większej tablicy nie jest jeszcze większą wiedzą, podobnie jak większy młotek nie jest automatycznie lepszym argumentem.

Agent AI może wesprzeć nauczyciela w odnalezieniu dowodów i porównaniu raportu z jawnymi kryteriami. Nie wystawia samodzielnie oceny. Ostateczną decyzję podejmuje nauczyciel, który może poprosić o wyjaśnienie dowolnego fragmentu pracy. Materiał przygotowany z pomocą AI, którego nie potraficie zweryfikować lub obronić, nie jest dowodem Waszych umiejętności i może drastycznie obniżyć ocenę, aż do oceny niedostatecznej.

🚧 Typowe pułapki

- nazwanie tablicy utworzonej przez `new[]` statyczną;
- założenie, że każda lokalna tablica ma statyczny czas przechowywania;
- uznanie pojęć „zmienna globalna” i „statyczny czas przechowywania” za synonimy;
- pominięcie ponownego przygotowania globalnej tablicy przed następną próbą;
- użycie `delete` zamiast `delete[]`;
- ręczne zwalnianie pamięci należącej do `std::unique_ptr` lub `std::vector`;
- sprawdzenie tylko zakresu wartości zamiast dokładnego wyniku;
- objęcie jednym pomiarem alokacji, przygotowania danych i przetwarzania;
- porównanie różnych danych lub różnej liczby operacji;
- wyciągnięcie ogólnego wniosku z pojedynczego czasu;
- stwierdzenie „sterta jest zawsze wolniejsza” bez dowodu;
- stwierdzenie „x64 jest zawsze szybsze” na podstawie samej nazwy;
- potraktowanie adresów z jednego uruchomienia jako gwarantowanej mapy pamięci;
- umieszczenie danych osobowych na zrzucie ekranu;
- oddanie kodu lub wniosków wygenerowanych przez AI bez ich sprawdzenia.

🏁 Co powinniście zapamiętać?

- Rozmiar tablicy, czas życia obiektu i miejsce przydzielenia pamięci to trzy powiązane, ale różne zagadnienia.
- Słowo `static` nie oznacza po prostu „tablica o rozmiarze wpisanym w kodzie”.
- Stos ułatwia automatyczne zarządzanie lokalnymi obiektami, ale ma ograniczony rozmiar.
- Pamięć dynamiczna daje elastyczność, lecz powinna mieć jasno określonego właściciela; w nowoczesnym C++ pomagają w tym RAII, `std::unique_ptr` i `std::vector`.
- Test funkcjonalny sprawdza dokładny wynik, a nie tylko brak awarii albo szeroki zakres wartości.
- Koszt alokacji trzeba oddzielić od kosztu przetwarzania już przydzielonych danych.
- Zmienna globalna opisuje zakres nazwy, a statyczny czas przechowywania opisuje czas istnienia obiektu; pojęcia są powiązane, ale nie są synonimami.
- Podobny czas trzech tablic jest pełnoprawnym i sensownym wynikiem eksperymentu.
- Raport ma pozwalać człowiekowi i wspierającemu go agentowi odnaleźć konkretne dowody, ale końcową ocenę oraz odpowiedzialność zachowuje człowiek.